

# Cartridge & Cloud — Vertical Slice Specification

VRM Games / Blas Luis Rocha González

2026-07-06

## Cartridge & Cloud — Vertical Slice Specification

### 0. Propósito, autoridad y uso

Este documento define el **contrato completo de alcance, integración, presentación, validación y cierre** del primer vertical slice de *Cartridge & Cloud*.

No es una lista de deseos, un resumen comercial ni una descripción genérica del proyecto. Debe utilizarse para responder, con evidencia verificable, a las siguientes preguntas:

- ¿Qué experiencia integrada debe poder completar una persona?
- ¿Qué sistemas forman parte del vertical slice?
- ¿Qué capacidades se consideran ya implementadas según la baseline documental?
- ¿Qué deuda representativa, de integración, rendimiento o QA continúa abierta?
- ¿Qué requisitos son bloqueantes?
- ¿Qué pruebas y evidencias deben producirse?
- ¿Qué defectos impiden aprobar el slice?
- ¿Qué condiciones deben cumplirse antes de iniciar empleados, investigación, puestos informáticos u otras expansiones?

El documento debe ser leído como un **contrato de producto interno**. Una capacidad solo puede considerarse cerrada cuando cumple simultáneamente su comportamiento, sus invariantes, su presentación mínima, sus pruebas, su integración, su trazabilidad y, cuando corresponda, su ejecución en una build externa.

#### 0.1. Jerarquía de autoridad

La consolidación utiliza esta jerarquía:

1. 00\_Enfoque\_y\_Alcance.md, como marco rector de visión, pilares y límites.
2. 01\_Game\_Design\_Document.md, como definición consolidada de reglas y sistemas.

3. `Vertical_Slice_Specification_v0.4`, como autoridad sobre el estado más reciente, el Golden Path, la escena representativa y los gates.
4. `Vertical_Slice_Specification_v0.3`, como registro de transición desde la fundación técnica hacia el bucle completo.
5. `Vertical_Slice_Specification_v0.2`, como fuente detallada de condiciones de entrada, aceptación, pruebas y Definition of Done.
6. `Vertical_Slice_Specification_v0.1`, como fuente de la intención original y de los primeros criterios de validación.

Cuando exista una contradicción:

- prevalece el documento de mayor autoridad;
- la tienda inicial vigente es la escena autorada de aproximadamente  $10 \times 15$  m, no la tienda histórica de  $10 \times 10$  celdas;
- el contenido representativo vigente se basa en seis productos iniciales y ocho familias de mobiliario, no en el antiguo mínimo de doce productos y seis familias;
- el bucle funcional se considera implementado según la baseline v0.6, pero eso no equivale a que el slice esté aprobado;
- las cifras económicas, temporales o de capacidad heredadas se consideran parámetros configurables hasta su validación;
- la descripción detallada de sistemas diferidos no los introduce en el vertical slice.

## 0.2. Relación con otros documentos

- `00_Enfoque_y_Alcance.md` define **por qué** existe el slice y qué identidad debe proteger.
- `01_Game_Design_Document.md` define **cómo deben comportarse** los sistemas.
- Este documento define **qué debe estar integrado y demostrado para cerrar el slice**.
- El TDD definirá la arquitectura y los contratos técnicos que permiten cumplirlo.
- El modelo de datos definirá estructuras, identificadores, catálogos, snapshots y relaciones.
- UX Flow detallará flujos, pantallas, estados de interfaz y feedback.
- QA Plan y QA Matrix convertirán estos criterios en ejecución sistemática y trazable.
- Roadmap y Sprint Plan asignarán el trabajo necesario para superar los gates.

## 0.3. Regla de interpretación del estado

La baseline v0.6 afirma que el bucle funcional está implementado y automatizado. Este documento conserva esa afirmación como fotografía documental fechada, pero exige evidencia de integración final.

Los estados utilizados son:

| Estado                   | Significado                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| IMPLEMENTADO             | Existe código e integración funcional según la baseline v0.6. Debe volver a validarse en la escena y build objetivo. |
| EN INTEGRACIÓN           | La capacidad existe, pero su integración representativa o su recorrido final no está cerrado.                        |
| PENDIENTE DE VALIDACIÓN  | La implementación puede existir, pero falta evidencia suficiente para aprobarla.                                     |
| PENDIENTE REPRESENTATIVO | Falta arte, audio, composición, iluminación, feedback o sustitución de placeholders.                                 |
| BLOQUEANTE               | Su fallo impide aprobar el gate o el vertical slice.                                                                 |
| DIFERIDO                 | Está fuera del vertical slice y no puede añadirse para “mejorar” el cierre.                                          |
| HIPÓTESIS CONFIGURABLE   | Valor de referencia sujeto a balance sin cambiar la intención del sistema.                                           |

---

## 1. Objetivo del vertical slice

### 1.1. Objetivo principal

Demostrar, mediante una **build interna estable y reproducible**, que la fantasía de construir y operar físicamente una tienda de videojuegos funciona como experiencia integrada.

La demostración debe comenzar en Bootstrap, permitir crear o cargar una partida, entrar en `StoreInitial.unity`, realizar el bucle comercial, cerrar una jornada, revisar resultados, guardar, salir y recargar sin pérdida ni contradicción de estado.

### 1.2. Experiencia que debe demostrar

Una persona que no dependa de herramientas de depuración debe poder:

1. entender dónde se encuentra y qué objetivo inmediato tiene;
2. preparar la tienda antes de abrir;
3. adquirir producto y, cuando corresponda, mobiliario;
4. recibir una entrega exactamente una vez;

5. trasladar unidades a ubicaciones válidas;
6. colocar o utilizar muebles sin bloquear accesos obligatorios;
7. asignar productos a displays compatibles;
8. reponer unidades visibles;
9. abrir la tienda;
10. observar al menos un cliente completar un recorrido comprensible;
11. comprobar que una unidad se reserva de forma segura;
12. atender la cola y confirmar una venta exactamente una vez;
13. cerrar la jornada sin dejar estados huérfanos;
14. revisar resultados económicos explicables;
15. guardar el estado;
16. salir y recargar;
17. verificar que dinero, inventario, precios, pedidos, muebles, día y demás estado relevante permanecen equivalentes.

### 1.3. Hipótesis de producto que debe validar

El slice debe aportar evidencia suficiente para aceptar o rechazar estas hipótesis:

1. **Organización física:** colocar y organizar mobiliario produce decisiones comprensibles y con consecuencias.
2. **Cadena comercial:** pedir, recibir, almacenar, asignar, reponer y vender forma un bucle satisfactorio.
3. **Clientes legibles:** el jugador puede comprender por qué un cliente busca, espera, compra o abandona.
4. **Economía recuperable:** el riesgo económico existe, pero un error temprano no destruye necesariamente la partida.
5. **Persistencia fiable:** una semana completa puede jugarse, guardarse y cargarse sin corrupción ni pérdida.
6. **Autoría espacial:** una tienda inicial autorada mejora lectura, identidad y confianza sin romper los sistemas lógicos.
7. **Escalabilidad:** el núcleo puede ampliarse con empleados, investigación y otros sistemas sin rehacer sus fundamentos.
8. **Control directo:** la operación física añade valor frente a una gestión exclusivamente mediante menús.
9. **Feedback suficiente:** la persona puede completar el recorrido sin observar logs o inspectors.
10. **Viabilidad productiva:** el alcance puede mantenerse, probarse y evolucionar con una producción principalmente individual.

### 1.4. No objetivos

El slice no pretende:

- representar el volumen final de contenido;
- demostrar todas las etapas empresariales;
- ser una demo pública;

- estar listo para comercialización;
  - contener arte o audio final en todas las áreas;
  - resolver todavía todos los problemas de balance de largo plazo;
  - incluir Steamworks, logros o Steam Cloud;
  - demostrar multijugador, modding o servicios reales;
  - sustituir una validación externa de mercado.
- 

## 2. Estado de referencia

### 2.1. Fotografía importada de la baseline v0.6

A fecha de la fuente v0.4 de esta especificación:

- versión de aplicación de referencia: **0.0.17**;
- Sprints 0–15: **CLOSED / PASS**;
- Sprint 16: **IN PROGRESS**;
- Sprint 17: **PENDING**;
- el bucle funcional está implementado y automatizado;
- la presentación representativa de la tienda inicial continúa en integración;
- el Golden Path manual, la suite completa, la build Windows x64 y el ejecutable externo forman parte del gate final.

Esta fotografía no debe confundirse con el estado vivo del repositorio después de la fecha indicada. Cualquier avance posterior deberá actualizar el documento operativo correspondiente y aportar evidencia.

#### 2.1.1. Actualización operativa vigente — 2026-07-06

- versión de aplicación y build de cierre de Sprint 16: **0.0.21**;
- Sprint 16: **COMPLETED / PASS**;
- aprobación visual y funcional de **StoreInitial**: **PASS**;
- compilación, **EditMode**, **PlayMode** y regresión manual: **PASS**;
- Golden Path, persistencia y **Player.log** en build Windows x64 externa: **PASS**;
- Sprint 17: **PENDING / READY TO OPEN**;
- **H6**: **BLOCKED / NOT RUN**;
- la Vertical Slice todavía no se declara validada hasta cerrar Sprint 17 y ejecutar el gate H6.

### 2.2. Matriz consolidada de madurez

| Área                                | Madurez de referencia | Trabajo exigido para el cierre                                                                                     |
|-------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------|
| Bootstrap, menú y slots             | IMPLEMENTADO          | Revalidar navegación completa, errores y retorno seguro.                                                           |
| Control del jugador                 | IMPLEMENTADO          | Revalidar en <code>StoreInitial.unity</code> con arquitectura y mobiliario finales del slice.                      |
| Cámara orbital y zoom               | IMPLEMENTADO          | Revalidar oclusiones, límites, interacción y consumo de input de UI.                                               |
| Grid, huellas, rotación y ocupación | IMPLEMENTADO          | Revalidar coincidencia entre preview, ocupación y visuales representativos.                                        |
| Validación de acceso                | IMPLEMENTADO          | Revalidar entradas, salida, caja, recepción y puntos obligatorios.                                                 |
| Productos e inventario              | IMPLEMENTADO          | Revalidar invariantes, transferencias y persistencia integrada.                                                    |
| Pedidos y recepción                 | IMPLEMENTADO          | Revalidar entrega única, cantidades, horarios y guardado.                                                          |
| Displays y reposición               | IMPLEMENTADO          | Revalidar compatibilidad, capacidad, asignación y stock visible.                                                   |
| Clientes y compra                   | IMPLEMENTADO          | Revalidar spawning, navegación, preferencias, paciencia y abandono.                                                |
| Reserva lógica temporal             | IMPLEMENTADO          | Revalidar exclusividad, liberación, cierre y recuperación.                                                         |
| Cola y checkout                     | IMPLEMENTADO          | Revalidar FIFO, preflight, venta atómica y feedback.                                                               |
| Ciclo diario                        | IMPLEMENTADO          | Revalidar <code>BeforeOpen</code> , <code>Open</code> , <code>Closing</code> , <code>Closed</code> y transiciones. |
| Economía y resultados               | IMPLEMENTADO          | Revalidar movimientos exactos, costes, impuestos y resumen.                                                        |

| Área                  | Madurez de referencia         | Trabajo exigido para el cierre                                                     |
|-----------------------|-------------------------------|------------------------------------------------------------------------------------|
| Persistencia          | IMPLEMENTADO                  | Revalidar tres slots, versionado, backup, autosave y equivalencia.                 |
| UI/UX integrada       | IMPLEMENTADO / EN INTEGRACIÓN | Cerrar input, feedback, errores, tutorial y legibilidad.                           |
| Tienda representativa | PENDIENTE REPRESENTATIVO      | Cerrar arquitectura, puerta, mobiliario, productos, displays, iluminación y audio. |
| Rendimiento objetivo  | PENDIENTE DE VALIDACIÓN       | Medir en perfil de hardware registrado y escena equipada.                          |
| Golden Path manual    | PENDIENTE DE VALIDACIÓN       | Ejecución completa sin herramientas de debug.                                      |
| Build Windows x64     | PENDIENTE DE VALIDACIÓN       | Development Build y ejecutable externo PASS.                                       |
| Aprobación del slice  | NO ALCANZADA                  | Requiere completar todos los gates de este documento.                              |

### 2.3. Principio de revalidación

Ninguna capacidad marcada como implementada queda exenta de prueba. La sustitución de escena, arquitectura, mobiliario, productos, materiales, colliders, navegación o UI puede introducir regresiones aunque el dominio permanezca correcto.

La validación final debe cubrir:

- comportamiento aislado;
- integración en `StoreInitial.unity`;
- recorrido Golden Path;
- persistencia antes y después de recargar;
- ejecución fuera del Editor;
- ausencia de regresiones en la suite completa.

## 3. Alcance funcional del vertical slice

### 3.1. Sistemas incluidos

El vertical slice incluye:

- Bootstrap y navegación de aplicación;
- creación, selección y carga de slots;
- entrada en la tienda inicial;
- movimiento por clic;
- cámara orbital de 360° y zoom;
- separación de input entre UI y mundo;
- construcción sobre grid de 0,5 m;
- preview, rotación de 90°, huella y ocupación;
- validación de límites, solapamiento, zonas y accesos;
- adquisición y colocación de mobiliario incluido;
- catálogo representativo de productos;
- proveedores y pedidos;
- transporte y recepción;
- inventario con ubicaciones explícitas;
- transferencias atómicas;
- asignación de productos a displays;
- reposición;
- precio de venta configurable;
- spawning y comportamiento de clientes;
- preferencias, paciencia, búsqueda y abandono;
- reserva lógica temporal de unidades;
- carrito o intención de compra;
- cola FIFO;
- checkout manual o iniciado por el jugador según el flujo vigente;
- venta atómica;
- estados del día;
- apertura y cierre;
- economía diaria y semanal;
- resumen de resultados;
- fiscalidad incluida por la implementación vigente;
- guardado manual en estados seguros;
- autosave tras el resumen cuando corresponda;
- tres slots;
- backup, versionado y recuperación incluidos por la persistencia vigente;
- UI necesaria para completar el recorrido;
- tutorial o guía suficiente para ejecutar el Golden Path;
- localización preparada para ES/EN en el contenido incluido;
- arte, iluminación y audio representativos;
- pruebas automatizadas, integración manual y build interna Windows x64.



### 3.2. Alcance representativo

El cierre exige una presentación suficientemente cercana al objetivo visual para evaluar la experiencia real. No basta con que el dominio funcione bajo placeholders.

Debe existir:

- una tienda inicial autorada;
- arquitectura correctamente alineada y escalada;
- puerta funcional y visualmente coherente;
- zona de entrada legible;
- mostrador y checkout reconocibles;
- recepción y almacén reconocibles;
- mobiliario inicial colocado en posiciones aprobadas;
- productos visibles en displays;
- materiales coherentes;
- iluminación interior representativa;
- audio funcional mínimo;
- ausencia de placeholders visibles no aprobados;
- feedback de interacción suficiente;
- UI coherente con la identidad vigente;
- legibilidad desde la cámara de juego.

### 3.3. Alcance temporal

La prueba extensa debe permitir completar al menos siete días simulados. El Golden Path mínimo puede completarse en una jornada, pero la aprobación requiere comprobar:

- repetición del bucle;
- transición entre días;
- persistencia acumulada;
- economía semanal;
- impuestos o cierre fiscal cuando corresponda;
- degradación o acumulación de errores;
- recuperación tras guardar y cargar en días distintos.

### 3.4. Parámetros de referencia configurables

Los siguientes valores se conservan como referencia de validación y deberán residir en datos cuando sea razonable:

- capital inicial: 20.000 €;
- mobiliario inicial: Tier E;
- proveedor inicial: uno general;
- empleados: ninguno;
- clientes simultáneos de referencia: hasta 8;

- arquetipos de cliente de referencia: 4;
- velocidades temporales heredadas: 5;
- apertura de referencia: 08:00;
- cierre manual de referencia: desde 20:00;
- cierre obligatorio de referencia: 22:00;
- rango histórico de precio: 50–150 % del recomendado;
- slots de guardado: 3;
- idiomas preparados: ES/EN;
- duración de validación: siete días o más.

Modificar un valor no reduce la cobertura exigida. Cualquier cambio que altere la experiencia deberá registrarse.

---

## 4. Contexto inicial y contenido representativo

### 4.1. Escena inicial

La escena objetivo es `StoreInitial.unity`.

Características vigentes:

- superficie aproximada: 10 × 15 m;
- grid lógico: 20 × 30 celdas de 0,5 m;
- arquitectura fija autorada;
- el runtime registra y opera referencias explícitas;
- la arquitectura no se reconstruye a partir de nombres, bounds, escalas o jerarquías accidentales de FBX;
- las referencias funcionales utilizan IDs o vínculos estables;
- la escena puede cambiar visualmente sin redefinir la verdad lógica.

### 4.2. Zonas funcionales mínimas

La tienda debe comunicar y soportar:

- entrada y salida;
- área de circulación inicial;
- zona de exposición;
- mostrador y checkout;
- recepción de mercancía;
- almacén;
- puntos de interacción;
- zonas colocables y no colocables;
- límites de navegación;
- reserva de acceso obligatoria;
- posiciones de cámara y validación necesarias.

### 4.3. Contenido mínimo vigente

El contenido representativo de referencia incluye:

- una tienda;
- una escena jugable principal;
- un proveedor general;
- seis productos iniciales;
- ocho familias de mobiliario representativas;
- cuatro arquetipos de cliente como referencia heredada;
- mobiliario suficiente para recepción, almacenamiento, exposición y checkout;
- un conjunto de materiales, iluminación y audio suficiente para evaluar la dirección visual;
- UI completa para el recorrido;
- tres slots;
- textos preparados para ES/EN en el alcance incluido.

La cifra de seis productos no impide crear variantes visuales o datos auxiliares. El requisito es que cada producto incluido tenga un rol verificable y participe en pedidos, inventario, display, cliente, precio, venta y persistencia.

### 4.4. Regla de representatividad

Un asset se considera representativo cuando:

- tiene escala coherente;
- no comunica una función equivocada;
- sus colliders y puntos funcionales coinciden con la lectura visual;
- puede observarse desde la cámara objetivo;
- usa materiales y sombreado compatibles con la dirección artística;
- no depende de un nombre temporal para su lógica;
- puede sustituirse sin romper IDs, datos o persistencia.

---

## 5. Golden Path obligatorio

### 5.1. Recorrido principal

Arrancar desde Bootstrap

- crear o seleccionar slot
- cargar o iniciar sesión
- entrar en StoreInitial
- inspeccionar estado inicial
- comprar producto y/o mobiliario incluido
- confirmar pedido
- recibir entrega exactamente una vez

- trasladar unidades a recepción/almacén
- colocar o utilizar mobiliario válido
- asignar producto a display
- reponer unidades visibles
- revisar o ajustar precio
- abrir tienda
- observar entrada de clientes
- cliente encuentra producto y reserva unidad
- cliente entra en cola
- checkout valida la venta
- consumir unidad y registrar ingreso de forma atómica
- cerrar tienda
- resolver clientes y reservas restantes
- revisar resultados
- guardar
- volver a menú o salir
- recargar slot
- verificar equivalencia de estado

## 5.2. Condiciones del Golden Path

El recorrido debe completarse:

- sin abrir inspectors;
- sin comandos de consola;
- sin modificar ScriptableObjects durante la ejecución;
- sin reposicionar objetos desde el Editor;
- sin inyectar dinero o stock mediante debug;
- sin saltar estados del día;
- sin limpiar manualmente datos de guardado entre pasos;
- sin ignorar errores del `Player.log`;
- con instrucciones disponibles dentro del producto o en un guion de prueba reproducible.

## 5.3. Variantes obligatorias

Además del recorrido feliz, se validarán al menos estas variantes:

- nueva partida;
- carga de partida existente;
- intento de colocación inválida;
- falta de dinero;
- falta de stock;
- pedido en condición límite;
- producto incompatible con display;
- cliente que abandona;
- cola saturada;

- cierre manual;
  - cierre forzado;
  - guardado y carga después de varios días;
  - backup o recuperación ante archivo principal no válido, si la implementación lo soporta;
  - ejecución externa fuera del Editor.
- 

## 6. Requisitos por sistema

### 6.1. Bootstrap, menú y slots

El flujo de aplicación debe:

- iniciar sin excepciones bloqueantes;
- presentar opciones comprensibles;
- permitir crear una partida sin datos previos;
- mostrar los tres slots y su estado;
- impedir cargar un slot inexistente como si fuera válido;
- manejar datos incompatibles o corruptos de forma controlada;
- entrar en la escena correcta;
- permitir volver a menú o cerrar la aplicación sin perder estado confirmado;
- distinguir claramente entre nueva partida, continuar y sobrescribir;
- evitar dobles confirmaciones por input repetido.

### 6.2. Input, movimiento y cámara

El control debe garantizar:

- clic de mundo solo cuando la UI no consume el evento;
- raycast a superficies válidas;
- destino accesible o feedback de fallo;
- ruta que respeta obstáculos y límites;
- ausencia de desplazamientos a través de mobiliario;
- recuperación controlada si una ruta deja de ser válida;
- cámara orbital estable;
- zoom dentro de límites configurados;
- legibilidad de puntos de interacción;
- ausencia de acciones de mundo al pulsar botones, sliders, listas o campos;
- consistencia entre cursor, highlight, preview y acción.

### 6.3. Construcción, grid y acceso

La construcción debe:

- usar celdas de 0,5 m como verdad lógica;
- resolver huella según orientación;

- rotar en incrementos de 90°;
- mostrar preview antes de confirmar;
- distinguir posición válida e inválida sin depender solo del color;
- impedir solapamiento no autorizado;
- impedir salir de límites;
- respetar zonas permitidas;
- preservar acceso a entrada, salida y puntos obligatorios;
- no aplicar mutaciones parciales ante fallo;
- registrar ocupación exactamente una vez;
- liberar exactamente la ocupación al mover o retirar;
- persistir posición, rotación, tipo e identidad;
- reconstruir el estado lógico tras cargar;
- mantener coherencia entre collider, visual, huella e interacción.

## 6.4. Productos e inventario

El inventario debe modelar ubicaciones explícitas. Como mínimo:

- recepción o entrega pendiente registrada;
- almacén;
- display o exposición;
- reserva lógica temporal;
- carrito o posesión lógica del cliente cuando corresponda;
- consumo por venta;
- cualquier ubicación adicional incluida por la implementación.

Invariante principal:

Stock registrado de un producto =  
 unidades en recepción  
 + unidades en almacén  
 + unidades en displays  
 + unidades reservadas válidas  
 + unidades en otras ubicaciones explícitas

Una unidad no puede:

- existir en dos ubicaciones;
- desaparecer durante una transferencia;
- aparecer sin movimiento registrado;
- venderse dos veces;
- permanecer reservada después de una cancelación, abandono o cierre;
- generar un ingreso sin consumo físico o lógico confirmado.

## 6.5. Proveedores, pedidos y recepción

Los pedidos deben:

- validar proveedor, producto, cantidad, precio y dinero;

- impedir cantidades inválidas;
- producir un coste o compromiso económico trazable;
- registrar un identificador estable;
- conservar estado pendiente, en tránsito, entregado o equivalente;
- llegar una sola vez;
- entregar exactamente la cantidad confirmada;
- manejar la hora límite conforme a datos;
- persistir antes y después de la entrega;
- impedir doble recepción por recarga, cambio de escena o repetición de evento;
- comunicar falta de dinero, producto no disponible o condición inválida.

La recepción debe:

- ser reconocible en la escena;
- disponer de capacidad o regla explícita;
- permitir traslado a almacén;
- impedir pérdida si el destino no puede aceptar unidades;
- resolver el cierre de día sin duplicación.

## 6.6. Displays, asignación y reposición

Cada display debe:

- tener identidad estable;
- declarar familia o compatibilidad;
- declarar capacidad;
- permitir una asignación válida de producto según las reglas vigentes;
- impedir asignaciones incompatibles;
- mostrar stock visible coherente;
- permitir reposición desde una ubicación autorizada;
- retirar unidades del origen solo si el destino puede aceptarlas;
- liberar o migrar unidades de forma segura al cambiar asignación;
- persistir asignación, capacidad ocupada y contenido;
- comunicar vacío, capacidad completa e incompatibilidad.

## 6.7. Precios y demanda

El sistema de precio debe:

- resolver un precio recomendado o base;
- permitir el rango configurado;
- impedir valores inválidos;
- aplicar cambios de forma determinista;
- afectar al interés o probabilidad de compra según reglas visibles en el GDD;
- persistir el precio;
- mostrar al jugador el precio actual y, cuando corresponda, la referencia;

- evitar que un cambio posterior altere una transacción ya confirmada;
- registrar ingresos con precisión monetaria adecuada.

El rango histórico 50–150 % se considera valor de referencia configurable, no una obligación codificada de forma rígida.

## 6.8. Clientes

Los clientes deben:

- aparecer únicamente durante estados permitidos;
- tener perfil o arquetipo válido;
- seleccionar objetivos compatibles con disponibilidad y preferencias;
- navegar sin atravesar obstáculos;
- reaccionar a falta de stock, precio, espera y cierre;
- evitar estados infinitos;
- reservar una unidad antes de comprometer compra;
- liberar la reserva al abandonar o fallar;
- entrar en cola cuando procede;
- completar compra o salir con causa comprensible;
- abandonar el mapa sin quedar registrados como activos;
- recibir limpieza segura al cerrar el día.

Su comportamiento debe poder comprenderse mediante animación, iconografía, texto o feedback. No puede depender de inspeccionar estados internos.

## 6.9. Reserva lógica temporal

Debe distinguirse entre:

- **reserva lógica temporal de inventario**, incluida en el núcleo para impedir dobles ventas;
- **servicio comercial de reservas anticipadas**, fuera del vertical slice.

La reserva lógica debe:

- ser exclusiva;
- asociar producto, unidad o cantidad y cliente/intención;
- tener estado y ciclo de vida explícitos;
- liberarse al abandonar, cancelar, fallar checkout o cerrar;
- consumirse una sola vez al vender;
- persistirse o resolverse de forma segura según el estado de guardado permitido;
- no convertir una intención en ingreso antes del checkout.

## 6.10. Cola y checkout

La cola debe:



- mantener un orden determinista, preferentemente FIFO salvo regla documentada;
- disponer de puntos reconocibles;
- actualizarse al entrar, avanzar, abandonar o cerrar;
- evitar superposición inaceptable;
- tener timeout o resolución ante bloqueo;
- liberar posiciones al completar una transacción.

El checkout debe realizar un preflight que confirme:

- cliente válido;
- reserva válida;
- unidad disponible en la ubicación esperada;
- precio resuelto;
- estación disponible;
- estado del día permitido;
- ausencia de transacción previa para la misma venta.

Después del preflight, la confirmación debe ser atómica:

1. consumir exactamente la unidad reservada;
2. registrar exactamente un ingreso;
3. liberar reserva y posición de cola;
4. actualizar métricas;
5. emitir feedback;
6. impedir repetición.

## 6.11. Ciclo diario

El ciclo debe usar estados explícitos equivalentes a:

- `BeforeOpen`;
- `Open`;
- `Closing`;
- `Closed`.

Debe garantizar:

- preparación antes de abrir;
- spawning solo en estado válido;
- cierre manual dentro de la ventana permitida;
- cierre obligatorio cuando corresponda;
- detención de nuevas entradas durante `Closing`;
- resolución segura de clientes activos;
- liberación de reservas;
- cierre de transacciones pendientes conforme a regla;
- generación única de resultados;
- avance único de día;
- ausencia de dobles cierres;

- persistencia coherente.

## 6.12. Economía y resultados

Todo cambio de saldo debe corresponder a un movimiento registrado.

El sistema debe distinguir, como mínimo:

- saldo;
- costes de pedido;
- costes operativos incluidos;
- ingresos por ventas;
- resultado diario;
- acumulación semanal;
- impuestos cuando corresponda;
- movimientos anulados o rechazados sin efecto.

Reglas:

- no puede existir dinero creado por repetir UI o eventos;
- no puede cobrarse dos veces una venta;
- un pedido rechazado no debe descontar dinero;
- el impuesto se aplica únicamente a beneficio semanal positivo según la regla vigente;
- el resumen debe explicar los principales cambios;
- la precisión monetaria debe evitar errores acumulativos visibles;
- guardar y cargar debe conservar el libro económico incluido.

## 6.13. Persistencia

La persistencia debe cubrir:

- tres slots;
- schema o versión;
- identificadores estables;
- dinero;
- día, hora y estado fiscal;
- inventario y ubicaciones;
- pedidos y entregas;
- precios;
- mobiliario, posición, rotación y ocupación;
- asignaciones y contenido de displays;
- progresión incluida;
- configuración necesaria para reconstruir la sesión;
- cualquier estado transitorio permitido en un punto de guardado.

Debe existir:

- guardado manual únicamente en estados seguros;

- autosave después del resumen cuando corresponda;
- escritura controlada;
- backup o mecanismo de recuperación incluido por la implementación;
- manejo de archivo inexistente, vacío, corrupto o incompatible;
- comparación antes/después;
- ausencia de duplicación al recargar.

## 6.14. UI, tutorial y accesibilidad base

La UI debe permitir completar el Golden Path sin herramientas externas.

Debe incluir o cubrir:

- navegación de menú;
- slots;
- HUD de dinero, hora y estado de tienda;
- acceso a operaciones necesarias;
- catálogo o compra;
- pedidos;
- inventario;
- asignación y reposición;
- precios;
- apertura y cierre;
- resultados;
- guardado;
- errores y confirmaciones;
- tutorial o guía contextual;
- foco, cierre y retorno coherentes;
- bloqueo de input del mundo cuando la UI consume interacción;
- escalado y legibilidad a  $1920 \times 1080$  como referencia;
- no depender solo del color para estados críticos;
- preparación de textos ES/EN en el contenido incluido.

## 6.15. Arte, iluminación y audio representativos

El cierre no exige arte final completo, pero sí una representación suficiente para evaluar:

- escala;
- silueta;
- legibilidad;
- circulación;
- identidad;
- coherencia de materiales;
- contraste;
- oclusión;
- densidad visual;

- feedback de interacción.

La escena debe evitar:

- placeholders visibles no aprobados;
- meshes flotantes o interpenetrados;
- puertas incoherentes con navegación;
- colliders que contradicen la forma;
- iluminación que oculta zonas funcionales;
- audio ausente en acciones críticas cuando sea necesario para feedback;
- iconos o textos temporales incomprensibles.

## 6.16. Rendimiento y estabilidad

Objetivo heredado:

60 FPS a 1920 × 1080

con tienda equipada y hasta 8 clientes simultáneos

Para que el resultado sea válido debe registrarse:

- hardware;
- sistema operativo;
- resolución;
- calidad;
- tipo de build;
- escena y estado;
- número de clientes;
- mobiliario y productos presentes;
- FPS medio y percentiles o frame time cuando sea posible;
- memoria;
- tiempos de carga;
- duración de la prueba.

El objetivo de 60 FPS es provisional mientras no exista un perfil mínimo de hardware aprobado. Aun así, una regresión grave, stutter persistente o degradación acumulativa bloquea el cierre.

---

## 7. Registro de criterios de aceptación

| ID         | Área       | Criterio                                                                              | Evidencia mínima                         |
|------------|------------|---------------------------------------------------------------------------------------|------------------------------------------|
| VS-APP-001 | Aplicación | Bootstrap inicia y alcanza menú sin excepción bloqueante.                             | Build/Editor, log y captura.             |
| VS-APP-002 | Aplicación | Nueva partida crea un slot válido una sola vez.                                       | Prueba manual y comparación de archivos. |
| VS-APP-003 | Aplicación | Carga de slot entra en la sesión correcta sin duplicar inicialización.                | Golden Path y logs.                      |
| VS-APP-004 | Aplicación | Volver al menú o salir no pierde un guardado confirmado.                              | Prueba manual.                           |
| VS-SCN-001 | Escena     | <code>StoreInitial.unity</code> carga con referencias obligatorias válidas.           | Validación de escena y log.              |
| VS-SCN-002 | Escena     | Arquitectura, puerta, zonas y mobiliario están colocados conforme al layout aprobado. | Revisión visual y checklist.             |
| VS-SCN-003 | Escena     | No existen placeholders visibles no exceptuados.                                      | Capturas y revisión.                     |
| VS-SCN-004 | Escena     | Visuales, colliders, navegación y puntos funcionales no se contradicen.               | Recorrido manual.                        |

| ID         | Área         | Criterio                                                                             | Evidencia mínima                |
|------------|--------------|--------------------------------------------------------------------------------------|---------------------------------|
| VS-INP-001 | Input        | Un evento consumido por UI no dispara movimiento, colocación o interacción de mundo. | Pruebas de propagación.         |
| VS-INP-002 | Input        | Clic de mundo produce destino válido o feedback de rechazo.                          | Recorrido manual.               |
| VS-CAM-001 | Cámara       | Órbita y zoom permanecen dentro de límites y no bloquean el flujo.                   | Prueba manual.                  |
| VS-CAM-002 | Cámara       | La cámara mantiene legibles entrada, displays, cola y checkout.                      | Capturas de referencia.         |
| VS-MOV-001 | Movimiento   | El jugador alcanza todos los puntos accesibles obligatorios.                         | Ruta completa.                  |
| VS-MOV-002 | Movimiento   | El jugador no atraviesa arquitectura ni mobiliario.                                  | Pruebas de colisión/navegación. |
| VS-MOV-003 | Movimiento   | Un destino inaccesible no deja al jugador en estado indefinido.                      | Prueba negativa.                |
| VS-BLD-001 | Construcción | Preview coincide con posición, rotación y huella final.                              | Prueba de colocación.           |
| VS-BLD-002 | Construcción | No se confirma colocación fuera de límites o sobre celdas ocupadas.                  | Pruebas negativas.              |

| ID         | Área         | Criterio                                                   | Evidencia mínima            |
|------------|--------------|------------------------------------------------------------|-----------------------------|
| VS-BLD-003 | Construcción | No se bloquean entrada, salida o puntos obligatorios.      | Prueba de acceso.           |
| VS-BLD-004 | Construcción | Fallo de validación no consume dinero ni altera ocupación. | Comparación antes/después.  |
| VS-BLD-005 | Construcción | Mover o retirar libera exactamente las celdas ocupadas.    | Prueba automatizada/manual. |
| VS-BLD-006 | Construcción | Posición, rotación, tipo e ID persisten tras recarga.      | Save/load.                  |
| VS-INV-001 | Inventario   | La suma de ubicaciones coincide con el stock registrado.   | Test de invariante.         |
| VS-INV-002 | Inventario   | Una unidad solo ocupa una ubicación lógica.                | Test automatizado.          |
| VS-INV-003 | Inventario   | Transferencia inválida no modifica origen ni destino.      | Prueba negativa.            |
| VS-INV-004 | Inventario   | Transferencia válida conserva cantidad total.              | Prueba automatizada.        |
| VS-INV-005 | Inventario   | No existen pérdidas o duplicaciones tras siete días.       | Auditoría de partida.       |
| VS-ORD-001 | Pedidos      | Pedido inválido se rechaza sin coste.                      | Prueba negativa.            |

| ID         | Área     | Criterio                                                            | Evidencia mínima           |
|------------|----------|---------------------------------------------------------------------|----------------------------|
| VS-ORD-002 | Pedidos  | Pedido válido registra proveedor, producto, cantidad, coste e ID.   | Inspección funcional.      |
| VS-ORD-003 | Pedidos  | Cada pedido se entrega una sola vez.                                | Prueba de recarga/evento.  |
| VS-ORD-004 | Pedidos  | La entrega contiene exactamente las cantidades confirmadas.         | Conteo y log.              |
| VS-ORD-005 | Pedidos  | Pedido pendiente persiste y reanuda correctamente.                  | Save/load.                 |
| VS-DSP-001 | Displays | Asignación incompatible se rechaza con feedback.                    | Prueba negativa.           |
| VS-DSP-002 | Displays | Capacidad impide sobrellenado.                                      | Prueba límite.             |
| VS-DSP-003 | Displays | Reposición consume origen y aumenta destino de forma atómica.       | Comparación de inventario. |
| VS-DSP-004 | Displays | Visual de producto coincide con asignación y cantidad representada. | Revisión visual.           |
| VS-DSP-005 | Displays | Asignación y stock visible persisten.                               | Save/load.                 |
| VS-PRC-001 | Precios  | Precio inválido se rechaza.                                         | Prueba negativa.           |



| ID         | Área     | Criterio                                                                     | Evidencia mínima             |
|------------|----------|------------------------------------------------------------------------------|------------------------------|
| VS-PRC-002 | Precios  | Precio confirmado persiste y se utiliza en la venta.                         | Save/load y transacción.     |
| VS-PRC-003 | Precios  | Cambiar precio afecta interés según regla determinada/configurada.           | Prueba comparativa.          |
| VS-CUS-001 | Clientes | Clientes solo aparecen durante estados permitidos.                           | Prueba de estados.           |
| VS-CUS-002 | Clientes | Cliente selecciona objetivo compatible o abandona con causa.                 | Observación y log de prueba. |
| VS-CUS-003 | Clientes | Cliente navega sin atravesar obstáculos ni quedar bloqueado indefinidamente. | Saturación y recorrido.      |
| VS-CUS-004 | Clientes | Falta de stock, precio o espera producen reacción legible.                   | Pruebas de variantes.        |
| VS-RES-001 | Reservas | Una unidad no puede reservarse para dos clientes.                            | Test concurrente.            |
| VS-RES-002 | Reservas | Abandono, fallo o cierre liberan la reserva.                                 | Pruebas de resolución.       |
| VS-RES-003 | Reservas | Venta consume la reserva exactamente una vez.                                | Test de checkout.            |
| VS-QUE-001 | Cola     | El orden de atención es determinista.                                        | Prueba con varios clientes.  |

| ID         | Área     | Criterio                                                        | Evidencia mínima          |
|------------|----------|-----------------------------------------------------------------|---------------------------|
| VS-QUE-002 | Cola     | Posiciones se liberan y avanzan correctamente.                  | Observación.              |
| VS-QUE-003 | Cola     | Saturación no produce bloqueo permanente.                       | Stress funcional.         |
| VS-CHK-001 | Checkout | Preflight rechaza transacción inválida sin mutación.            | Test negativo.            |
| VS-CHK-002 | Checkout | Venta válida consume una unidad y registra un ingreso.          | Test automatizado.        |
| VS-CHK-003 | Checkout | La misma venta no puede confirmarse dos veces.                  | Doble input/evento.       |
| VS-CHK-004 | Checkout | Cliente, reserva, cola y métricas quedan resueltos tras vender. | Inspección postcondición. |
| VS-DAY-001 | Día      | Transiciones entre estados siguen el orden permitido.           | Test de state machine.    |
| VS-DAY-002 | Día      | Abrir habilita clientes una sola vez.                           | Prueba de apertura.       |
| VS-DAY-003 | Día      | Cerrar impide nuevas entradas y resuelve activos.               | Cierre manual/forzado.    |
| VS-DAY-004 | Día      | Resumen y avance se generan una sola vez.                       | Doble input/recarga.      |
| VS-ECO-001 | Economía | Todo cambio de saldo tiene movimiento registrado.               | Auditoría contable.       |
| VS-ECO-002 | Economía | Venta genera un único ingreso con precio correcto.              | Test de transacción.      |

| ID         | Área         | Criterio                                                        | Evidencia mínima              |
|------------|--------------|-----------------------------------------------------------------|-------------------------------|
| VS-ECO-003 | Economía     | Pedido rechazado no cambia saldo.                               | Prueba negativa.              |
| VS-ECO-004 | Economía     | Impuesto solo se aplica a beneficio semanal positivo.           | Casos positivo/cero/negativo. |
| VS-ECO-005 | Economía     | Resumen diario/semanal reconcilia movimientos.                  | Comparación.                  |
| VS-SAV-001 | Persistencia | Guardado conserva todos los campos obligatorios.                | Snapshot y diff.              |
| VS-SAV-002 | Persistencia | Carga reconstruye un estado equivalente.                        | Comparación antes/después.    |
| VS-SAV-003 | Persistencia | Cargar no duplica pedidos, muebles, stock, clientes o ingresos. | Prueba de repetición.         |
| VS-SAV-004 | Persistencia | Archivo ausente o corrupto se maneja de forma controlada.       | Prueba negativa/backup.       |
| VS-SAV-005 | Persistencia | Tres slots son independientes.                                  | Prueba cruzada.               |
| VS-UI-001  | UI/UX        | Golden Path puede completarse sin herramientas de debug.        | Sesión guiada/no guiada.      |
| VS-UI-002  | UI/UX        | Errores y estados inválidos muestran causa comprensible.        | Checklist de feedback.        |
| VS-UI-003  | UI/UX        | Controles críticos no dependen solo del color.                  | Revisión de accesibilidad.    |

| ID           | Área           | Criterio                                                                              | Evidencia mínima              |
|--------------|----------------|---------------------------------------------------------------------------------------|-------------------------------|
| VS-UI-004    | UI/UX          | Textos incluidos están preparados para ES/EN sin desbordes bloqueantes.               | Revisión de localización.     |
| VS-ART-001   | Representación | Escena cumple layout, escala y silueta aprobados.                                     | Revisión visual.              |
| VS-ART-002   | Representación | Iluminación permite leer zonas, productos, clientes y cola.                           | Capturas y recorrido.         |
| VS-AUD-001   | Audio          | Acciones críticas incluidas tienen feedback sonoro o alternativa aprobada.            | Checklist.                    |
| VS-PER-001   | Rendimiento    | Prueba objetivo registra 60 FPS/1080p o desviación justificada en perfil documentado. | Captura de profiler/medición. |
| VS-PER-002   | Rendimiento    | No existe degradación acumulativa grave durante la sesión extensa.                    | Soak test.                    |
| VS-QA-001    | QA             | Suite automatizada completa permanece verde.                                          | Reporte de tests.             |
| VS-QA-002    | QA             | Golden Path manual obtiene PASS reproducible.                                         | Acta de ejecución.            |
| VS-BUILD-001 | Build          | Windows x64 Development Build se genera sin error.                                    | Artefacto y log.              |

| ID           | Área          | Criterio                                                               | Evidencia mínima    |
|--------------|---------------|------------------------------------------------------------------------|---------------------|
| VS-BUILD-002 | Build         | Ejecutable arranca y completa Golden Path fuera del Editor.            | Acta externa.       |
| VS-BUILD-003 | Build         | <code>Player.log</code> no contiene errores bloqueantes o recurrentes. | Log archivado.      |
| VS-DOC-001   | Documentación | Resultados, defectos, excepciones y decisiones están trazados.         | Matriz y changelog. |

### 7.1. Regla de resultado

Cada criterio debe marcarse como:

- PASS;
- FAIL;
- BLOCKED;
- NOT RUN;
- EXCEPTION APPROVED.

EXCEPTION APPROVED solo es válido cuando existe:

- defecto o desviación identificado;
- impacto analizado;
- responsable de decisión;
- justificación;
- mitigación;
- fecha o condición de revisión;
- confirmación de que no oculta un S0/S1.

## 8. Plan de pruebas de aceptación

### 8.1. Niveles de prueba

#### Nivel A — Pruebas deterministas

Cubren dominio, aplicación, invariantes y transiciones sin depender de representación visual cuando sea posible.

### Nivel B — Integración en TestLab

Cubren componentes, adaptadores, escenas reducidas y regresión rápida.

### Nivel C — Integración en StoreInitial

Cubren la escena representativa, referencias, navegación, input, UI y composición real.

### Nivel D — Golden Path manual

Cubre la experiencia completa sin herramientas de debug.

### Nivel E — Build externa

Cubre generación Windows x64, arranque, recorrido y logs fuera del Editor.

### Nivel F — Sesión extensa

Cubre siete días, persistencia acumulada, economía semanal, estabilidad y degradación.

## 8.2. Escenarios obligatorios

| ID         | Escenario                              | Nivel | Ejecución                                     | Resultado esperado                      |
|------------|----------------------------------------|-------|-----------------------------------------------|-----------------------------------------|
| VS-TST-001 | Nueva partida desde instalación limpia | D     | Crear slot, entrar, completar primer día.     | Slot válido y recorrido sin bloqueo.    |
| VS-TST-002 | Tres slots independientes              | D     | Crear estados distintos en cada slot.         | No hay contaminación cruzada.           |
| VS-TST-003 | Golden Path completo                   | D/E   | Ejecutar todos los pasos oficiales.           | PASS sin debug.                         |
| VS-TST-004 | Siete días consecutivos                | F     | Completar semana con varias compras y ventas. | Sin pérdida, duplicación ni corrupción. |
| VS-TST-005 | Cierre manual                          | C/D   | Cerrar en ventana permitida.                  | Clientes y reservas resueltos.          |

| ID         | Escenario                  | Nivel | Ejecución                                   | Resultado esperado                |
|------------|----------------------------|-------|---------------------------------------------|-----------------------------------|
| VS-TST-006 | Cierre forzado             | C/D   | Alcanzar hora límite con clientes activos.  | Transición determinista y segura. |
| VS-TST-007 | Colocación válida          | B/C   | Colocar y rotar cada familia incluida.      | Preview y ocupación coinciden.    |
| VS-TST-008 | Colocación solapada        | B/C   | Intentar ocupar celdas usadas.              | Rechazo sin mutación.             |
| VS-TST-009 | Bloqueo de entrada         | B/C   | Intentar bloquear reserva obligatoria.      | Rechazo con causa.                |
| VS-TST-010 | Mover y retirar            | B/C   | Mover objeto y comprobar celdas.            | Liberación exacta.                |
| VS-TST-011 | Pedido sin dinero          | A/C   | Confirmar pedido superior al saldo.         | Rechazo sin coste.                |
| VS-TST-012 | Pedido válido              | A/C   | Comprar varias cantidades.                  | Registro correcto.                |
| VS-TST-013 | Entrega única tras recarga | A/C   | Guardar antes de entrega, cargar y avanzar. | Entrega una vez.                  |
| VS-TST-014 | Capacidad de recepción     | A/C   | Forzar destino insuficiente.                | Sin pérdida.                      |
| VS-TST-015 | Transferencia válida       | A/C   | Mover recepción→almacén→display.            | Conservación de disponibilidad.   |
| VS-TST-016 | Transferencia inválida     | A/C   | Destino incompatible/lleeno.                | Sin mutación parcial.             |
| VS-TST-017 | Asignación incompatible    | A/C   | Asignar producto no admitido.               | Rechazo y feedback.               |

| ID         | Escenario                      | Nivel | Ejecución                                 | Resultado esperado                |
|------------|--------------------------------|-------|-------------------------------------------|-----------------------------------|
| VS-TST-018 | Reposición límite              | A/C   | Reponer hasta capacidad.                  | No sobrellenado.                  |
| VS-TST-019 | Comparación de precios         | C/F   | Usar precio bajo, recomendado y alto.     | Respuesta de demanda observable.  |
| VS-TST-020 | Sin stock                      | C/D   | Abrir con producto demandado agotado.     | Abandono o alternativa legible.   |
| VS-TST-021 | Dos clientes, una unidad       | A/C   | Competencia por última unidad.            | Una sola reserva/venta.           |
| VS-TST-022 | Abandono libera reserva        | A/C   | Provocar espera o fallo.                  | Unidad vuelve a disponibilidad.   |
| VS-TST-023 | Cola saturada                  | C/F   | Alcanzar carga de clientes de referencia. | Sin deadlock.                     |
| VS-TST-024 | Doble confirmación de checkout | A/C   | Repetir input/evento.                     | Una sola venta.                   |
| VS-TST-025 | Cierre con reserva activa      | A/C   | Cerrar durante recorrido.                 | Resolución segura.                |
| VS-TST-026 | Semana con beneficio           | A/F   | Cerrar semana positiva.                   | Impuesto correcto.                |
| VS-TST-027 | Semana sin beneficio           | A/F   | Cerrar semana cero/negativa.              | Sin impuesto positivo indebido.   |
| VS-TST-028 | Guardar en estado seguro       | C/D   | Guardar cuando está permitido.            | Archivo válido.                   |
| VS-TST-029 | Guardar en estado no seguro    | C/D   | Intentar guardar donde no procede.        | Rechazo o resolución controlada.  |
| VS-TST-030 | Equivalencia de recarga        | A/D   | Snapshot antes y después.                 | Campos obligatorios equivalentes. |



| ID         | Escenario                  | Nivel | Ejecución                                           | Resultado esperado                |
|------------|----------------------------|-------|-----------------------------------------------------|-----------------------------------|
| VS-TST-031 | Archivo principal corrupto | B/E   | Probar recuperación soportada.                      | Mensaje y backup/control.         |
| VS-TST-032 | Input de UI                | C/D   | Operar todas las pantallas sobre mundo interactivo. | Sin propagación.                  |
| VS-TST-033 | Resolución 1080p           | C/E   | Recorrer UI completa.                               | Sin cortes bloqueantes.           |
| VS-TST-034 | Idioma ES                  | C/E   | Completar Golden Path.                              | Textos correctos.                 |
| VS-TST-035 | Idioma EN                  | C/E   | Completar Golden Path.                              | Sin desbordes bloqueantes.        |
| VS-TST-036 | Puerta y navegación        | C/E   | Entrada/salida con varios agentes.                  | Sin colisiones o bloqueo.         |
| VS-TST-037 | Sesión de rendimiento      | E/F   | Tienda equipada y 8 clientes.                       | Métricas registradas.             |
| VS-TST-038 | Soak test                  | E/F   | Sesión prolongada y varios días.                    | Sin degradación grave.            |
| VS-TST-039 | Build externa              | E     | Ejecutar en entorno fuera del Editor.               | Arranque, guardado y log válidos. |
| VS-TST-040 | Regresión completa         | A/B/C | Ejecutar suite y checklist.                         | Todo verde o excepción formal.    |

### 8.3. Datos de prueba

Los datos deberán cubrir:

- saldo suficiente e insuficiente;
- cantidades cero, mínimas, máximas y superiores a capacidad;
- productos compatibles e incompatibles;
- displays vacíos, parciales y llenos;
- clientes con preferencias distintas;
- precios bajos, recomendados y altos;
- pedidos antes y después de límites temporales;
- estados de guardado en distintos días;
- semana con beneficio, cero y pérdida;

- rutas despejadas y congestionadas;
- cola vacía y saturada.

## 8.4. Reproducibilidad

Toda incidencia debe incluir:

- versión o commit;
- escena;
- slot o datos iniciales;
- pasos exactos;
- resultado esperado;
- resultado real;
- frecuencia;
- severidad;
- captura o vídeo cuando aporte valor;
- logs;
- save asociado cuando sea posible.

## 9. Severidades, prioridades y política de defectos

### 9.1. Severidades

| Severidad                | Definición                                                                           | Ejemplos                                                                                | Efecto sobre el gate         |
|--------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------|
| <b>S0 — Catastrófico</b> | Impide ejecutar, destruye datos o invalida la prueba completa.                       | Build no arranca, corrupción general, pérdida masiva.                                   | Bloqueo inmediato.           |
| <b>S1 — Crítico</b>      | Rompe el Golden Path, una invariante o un sistema esencial sin workaround aceptable. | Duplicación de stock, doble cobro, no puede cerrar, save no carga.                      | No puede aprobarse.          |
| <b>S2 — Importante</b>   | Degrada seriamente la experiencia o presenta riesgo alto con workaround limitado.    | Cliente bloqueado frecuente, UI confusa en acción esencial, caída grave de rendimiento. | Resolver o excepción formal. |

| Severidad             | Definición                                               | Ejemplos                                                   | Efecto sobre el gate                        |
|-----------------------|----------------------------------------------------------|------------------------------------------------------------|---------------------------------------------|
| <b>S3 — Menor</b>     | Defecto visible con impacto limitado y workaround claro. | Desalineación leve, texto secundario, feedback no crítico. | Puede aceptarse con registro.               |
| <b>S4 — Cosmético</b> | No afecta función ni comprensión relevante.              | Pulido visual menor.                                       | No bloquea, debe documentarse si permanece. |

## 9.2. Política de aprobación

- Cero S0 abiertos.
- Cero S1 abiertos.
- Todo S2 debe resolverse o disponer de excepción formal.
- Los S3/S4 restantes deben estar registrados, priorizados y no acumular un problema sistémico.
- Una incidencia repetida que degrade el Golden Path puede elevarse de severidad.
- Un defecto visual puede ser S1/S2 si comunica una función falsa o impide interactuar.
- Un test inestable no se considera PASS.

## 10. Gate de Sprint 16 — Cierre representativo

### 10.1. Objetivo

Sustituir la presentación provisional por una tienda inicial representativa sin romper el bucle funcional.

### 10.2. Entradas

- bucle funcional integrado;
- layout y visual target aprobados;
- assets representativos disponibles;
- plan de autoría de `StoreInitial.unity`;
- IDs y referencias funcionales definidos;
- suite de regresión existente;
- backup o control de cambios suficiente para revertir.

### 10.3. Requisitos de salida

Sprint 16 solo puede cerrarse cuando:

- `StoreInitial.unity` contiene la arquitectura aprobada;
- la tienda mide aproximadamente  $10 \times 15$  m;
- puerta, entrada y circulación funcionan;
- recepción, almacén, displays y checkout son reconocibles;
- mobiliario inicial usa posiciones aprobadas;
- productos representativos aparecen correctamente;
- colliders, NavMesh y puntos funcionales coinciden con los visuales;
- no hay placeholders visibles salvo excepciones aprobadas;
- iluminación y audio alcanzan nivel representativo;
- la UI no propaga input al mundo;
- construcción y acceso siguen funcionando;
- inventario, pedidos, displays, clientes, checkout, día, economía y save/load no presentan regresiones S0/S1;
- la suite completa permanece verde;
- existe evidencia visual y funcional de revisión.

### 10.4. Evidencia de Sprint 16

- capturas de layout aprobado;
- vídeo o recorrido de escena;
- checklist de arquitectura;
- lista de referencias funcionales;
- reporte de tests;
- inventario de placeholders restantes;
- defectos abiertos y severidad;
- registro de cambios;
- decisión formal de PASS/FAIL.

### 10.5. Decisión formal de cierre — 2026-07-06

Sprint 16 queda **COMPLETED** / **PASS** para la build **0.0.21**. La decisión se apoya en:

- StoreInitial autorada y conectada mediante referencias explícitas;
- aprobación visual manual sin placeholders o contradicciones bloqueantes;
- aprobación funcional del Golden Path;
- compilación y suites EditMode/PlayMode en PASS;
- regresión manual en PASS;
- build Windows x64 externa iniciada desde Bootstrap;
- guardado, cierre, reapertura y carga equivalente en PASS;
- `Player.log` revisado sin errores bloqueantes;
- ausencia de defectos S0/S1 conocidos dentro del alcance de cierre;
- roadmap, producción, trazabilidad y signoff actualizados.

Esta decisión satisface el gate representativo de Sprint 16, pero no equivale a H6 ni a la aprobación completa de la Vertical Slice.

## 10.6. Causas de no cierre

- arquitectura generada o deducida de forma frágil;
  - puerta visualmente correcta pero funcionalmente inválida;
  - navegación rota;
  - colliders contradictorios;
  - mobiliario que bloquea el Golden Path;
  - productos no visibles o mal asignados;
  - input de UI con efectos en mundo;
  - regresión de persistencia;
  - placeholders críticos sin excepción;
  - falta de evidencia reproducible.
- 

## 11. Gate de Sprint 17 — Estabilización y build

### 11.1. Objetivo

Convertir el slice representativo en una build interna estable, medible y aprobable.

### 11.2. Áreas obligatorias

- corrección de defectos;
- balance inicial;
- rendimiento;
- estabilidad;
- QA completo;
- Golden Path manual;
- sesión de siete días;
- build Windows x64;
- ejecución externa;
- revisión de `Player.log`;
- documentación y trazabilidad.

### 11.3. Requisitos de salida

Sprint 17 solo puede cerrarse cuando:

- Sprint 16 está aprobado;
- todos los criterios bloqueantes tienen PASS;
- no existen S0/S1;
- los S2 tienen resolución o excepción formal;

- la suite automatizada completa está verde;
- el Golden Path manual obtiene PASS;
- la prueba de siete días obtiene PASS;
- save/load conserva equivalencia;
- no hay pérdida o duplicación de inventario;
- no hay doble transacción;
- el cierre diario es determinista;
- la prueba de rendimiento está documentada;
- la Windows x64 Development Build se genera;
- el ejecutable externo completa el recorrido requerido;
- `Player.log` no presenta errores bloqueantes o recurrentes;
- el paquete de evidencias está completo;
- existe informe de validación y decisión de gate.

#### 11.4. Regla de balance

No se exige balance final. Sí se exige que:

- la nueva partida sea jugable;
  - el capital inicial permita ejecutar el recorrido;
  - los precios produzcan diferencias observables;
  - los costes no generen un callejón sin salida inmediato sin advertencia;
  - la economía pueda explicar beneficio o pérdida;
  - el jugador pueda recuperarse de errores razonables;
  - no existan exploits obvios que invaliden la prueba.
- 

## 12. Definition of Done del vertical slice

El vertical slice se considera cerrado únicamente cuando se cumplen todos los puntos siguientes:

1. Todos los sistemas incluidos están implementados e integrados.
2. `StoreInitial.unity` está autorada y visualmente aprobada.
3. El Golden Path puede completarse sin debug.
4. La partida de siete días se completa.
5. Guardar, salir y cargar conserva un estado equivalente.
6. No existen pérdidas o duplicaciones de inventario.
7. Las reservas lógicas no se duplican ni quedan huérfanas.
8. Cada venta consume una unidad y registra un ingreso exactamente una vez.
9. La cola no entra en deadlock.
10. El cierre diario resuelve clientes, reservas y transacciones con seguridad.
11. La economía reconcilia todos los movimientos incluidos.
12. La UI no propaga input al mundo.
13. El tutorial o guía permite comprender el recorrido.

14. Arte, iluminación y audio alcanzan nivel representativo.
15. La suite automatizada completa está verde.
16. Todos los criterios bloqueantes tienen PASS.
17. No existen S0/S1 abiertos.
18. Todo S2 restante tiene excepción formal.
19. El rendimiento objetivo se ha medido y documentado.
20. La build Windows x64 se genera correctamente.
21. El ejecutable externo completa el recorrido requerido.
22. `Player.log` no contiene errores bloqueantes o recurrentes.
23. Los defectos restantes están registrados.
24. La documentación afectada está actualizada.
25. La trazabilidad conecta requisito, prueba, resultado y defecto.
26. Existe un informe de validación reproducible.
27. Existe una decisión formal de aprobación.

El cumplimiento parcial no equivale a cierre. “Funciona en el Editor” no sustituye a la build externa. “La suite está verde” no sustituye al Golden Path manual. “Se ve bien” no sustituye a invariantes y persistencia.

---

## 13. Paquete de evidencias

La aprobación debe conservar, como mínimo:

```
VerticalSlice_Evidence/  
├─ 00_Validation_Report.md  
├─ 01_Acceptance_Matrix.csv  
├─ 02_Test_Run_Summary.md  
├─ 03_Defect_Register.csv  
├─ 04_Exception_Register.md  
├─ 05_Golden_Path_Record.md  
├─ 06_Seven_Day_Run_Record.md  
├─ 07_Performance_Record.md  
├─ 08_Build_Record.md  
├─ 09_Player.log  
├─ 10_Save_Comparison/  
├─ 11_Screenshots/  
├─ 12_Videos/  
├─ 13_Test_Reports/  
└─ 14_Checksums.txt
```

### 13.1. Informe de validación

Debe incluir:

- versión;

- commit;
- fecha;
- responsable;
- hardware;
- sistema operativo;
- configuración;
- alcance ejecutado;
- criterios PASS/FAIL;
- defectos;
- excepciones;
- rendimiento;
- build;
- riesgos;
- decisión final.

### 13.2. Comparación de guardado

La evidencia de persistencia debe poder demostrar:

- qué estado existía antes de guardar;
- qué datos se escribieron;
- qué estado se reconstruyó;
- qué diferencias se detectaron;
- por qué una diferencia es esperada o defectuosa.

### 13.3. Integridad del paquete

Cuando se archive una baseline de cierre, los artefactos relevantes deben disponer de hash o mecanismo equivalente para evitar ambigüedad sobre qué build, logs y resultados fueron aprobados.

---

## 14. Fuera de alcance

Quedan fuera del vertical slice:

- empleados completos;
- sistema de investigación funcional;
- puestos informáticos;
- servicio comercial de reservas anticipadas;
- eventos complejos de tienda;
- ampliaciones extensas del complejo;
- comercio online;
- publishing;
- desarrollo interno;
- plataforma digital;



- infraestructura y servicios online;
- mercado competitivo completo;
- usados y retro avanzados;
- devoluciones complejas;
- robos;
- marketing avanzado;
- Steamworks;
- logros;
- Steam Cloud;
- workshop;
- modding;
- multijugador;
- demo pública;
- arte final completo;
- audio final completo;
- contenido masivo;
- balance comercial definitivo.

### 14.1. Regla anti-expansión

No se añadirá un sistema fuera de alcance para resolver un problema del slice cuando pueda resolverse estabilizando el núcleo.

Ejemplos:

- no añadir empleados para ocultar que las tareas manuales no son claras;
- no añadir investigación para compensar una progresión insuficiente de siete días;
- no añadir puestos informáticos para crear variedad antes de que vender sea satisfactorio;
- no añadir comercio online para evitar arreglar navegación o colas;
- no añadir contenido masivo para ocultar falta de feedback.

---

## 15. Condición para iniciar sistemas posteriores

Después de aprobar el slice, el proyecto podrá planificar sistemas diferidos solo cuando exista evidencia de que:

- el bucle base es comprensible;
- la construcción produce decisiones;
- la cadena de inventario es estable;
- los clientes son legibles;
- la economía permite riesgo y recuperación;
- la persistencia es fiable;
- la escena representativa no rompe el dominio;

- la producción de contenido es sostenible;
- la arquitectura admite extensión sin acoplamiento frágil;
- los principales defectos están controlados.

La decisión sobre el primer sistema posterior deberá basarse en playtests y coste.  
El orden conceptual recomendado es:

1. pulido y expansión del contenido base;
2. mejoras y ampliaciones de tienda;
3. empleados;
4. investigación;
5. puestos informáticos;
6. comercio online y logística.

Este orden no es automático. Debe revisarse contra resultados reales.

---

## 16. Gobierno y control de cambios

### 16.1. Cambio menor

Puede tratarse como ajuste menor:

- corregir texto;
- mejorar feedback;
- ajustar un valor configurable;
- sustituir un asset sin alterar función;
- añadir una prueba;
- aclarar un criterio sin cambiar alcance.

### 16.2. Cambio mayor

Requiere aprobación formal:

- añadir o eliminar un sistema del slice;
- cambiar el Golden Path;
- reducir una invariante;
- eliminar build externa;
- cambiar la tienda inicial;
- modificar contenido mínimo de forma que reduzca cobertura;
- aceptar un S1;
- adelantar un sistema diferido;
- cambiar plataforma, motor o modelo de control;
- declarar cerrado el slice sin una evidencia exigida.

### 16.3. Registro mínimo

Todo cambio mayor deberá indicar:

- motivación;
- fuente;
- impacto en enfoque y GDD;
- impacto técnico;
- impacto en QA;
- impacto en producción;
- criterios modificados;
- decisión;
- fecha;
- responsable.

## 17. Matriz de trazabilidad de alto nivel

| Capacidad           | Enfoque                             | GDD                      | Criterios principales  | Gate         |
|---------------------|-------------------------------------|--------------------------|------------------------|--------------|
| Autoría de tienda   | Gestión visible / autoría explícita | Mundo y espacio          | VS-SCN-001...004       | Sprint 16    |
| Movimiento y cámara | Claridad y control                  | Control                  | VS-INP, VS-CAM, VS-MOV | Sprint 16/17 |
| Construcción        | Construcción funcional              | Grid y ocupación         | VS-BLD-001...006       | Sprint 16/17 |
| Inventario          | Sistemas comprensibles              | Stock y transferencias   | VS-INV-001...005       | Sprint 17    |
| Pedidos             | Gestión visible                     | Proveedores y recepción  | VS-ORD-001...005       | Sprint 17    |
| Displays            | Gestión visible                     | Asignación y reposición  | VS-DSP-001...005       | Sprint 16/17 |
| Clientes            | Clientes observables                | Estados y comportamiento | VS-CUS-001...004       | Sprint 17    |
| Reservas            | Sistemas comprensibles              | Exclusividad de unidad   | VS-RES-001...003       | Sprint 17    |
| Checkout            | Claridad / atomicidad               | Venta                    | VS-QUE / VS-CHK        | Sprint 17    |
| Día y economía      | Progresión comprensible             | Ciclo y resultados       | VS-DAY / VS-ECO        | Sprint 17    |
| Persistencia        | Calidad                             | Save/load                | VS-SAV-001...005       | Sprint 17    |

| Capacidad         | Enfoque            | GDD              | Criterios principales | Gate         |
|-------------------|--------------------|------------------|-----------------------|--------------|
| UI/UX             | Claridad y control | Flujos           | VS-UI-001...004       | Sprint 16/17 |
| Representación    | Gestión visible    | Dirección visual | VS-ART / VS-AUD       | Sprint 16    |
| Rendimiento/buena | Calidad            | Cierre técnico   | VS-PER / VS-BUILD     | Sprint 17    |

## Anexo A. Checklist de ejecución del Golden Path

- [ ] Bootstrap inicia sin error bloqueante.
- [ ] Menú muestra slots correctamente.
- [ ] Se crea o carga el slot objetivo.
- [ ] StoreInitial carga con referencias válidas.
- [ ] Jugador y cámara responden.
- [ ] UI no dispara acciones de mundo.
- [ ] Se adquiere producto.
- [ ] Se adquiere o coloca mobiliario cuando procede.
- [ ] Pedido queda registrado.
- [ ] Entrega ocurre una sola vez.
- [ ] Cantidades recibidas son correctas.
- [ ] Unidades pasan a almacén sin pérdida.
- [ ] Display acepta producto compatible.
- [ ] Reposición conserva la cantidad total.
- [ ] Precio se muestra y se aplica.
- [ ] Tienda abre una sola vez.
- [ ] Clientes aparecen y navegan.
- [ ] Un cliente reserva una unidad.
- [ ] Cliente entra en cola.
- [ ] Checkout confirma una venta.
- [ ] Se consume una única unidad.
- [ ] Se registra un único ingreso.
- [ ] Cliente y reserva quedan resueltos.
- [ ] Cierre detiene nuevas entradas.
- [ ] Clientes restantes se resuelven.
- [ ] Resumen se genera una vez.
- [ ] Estado se guarda.
- [ ] Se vuelve a menú o se cierra.
- [ ] Slot se recarga.
- [ ] Dinero coincide.
- [ ] Inventario coincide.

- [ ] Pedidos coinciden.
- [ ] Precios coinciden.
- [ ] Mobiliario y ocupación coinciden.
- [ ] Día y estado fiscal coinciden.
- [ ] No existen errores bloqueantes en log.

## Anexo B. Checklist de tienda representativa

- [ ] Dimensión aproximada 10 × 15 m.
- [ ] Grid lógico 20 × 30 coherente.
- [ ] Arquitectura autorada, no generada desde FBX.
- [ ] Entrada y salida legibles.
- [ ] Puerta visual y funcionalmente correcta.
- [ ] Reserva de acceso preservada.
- [ ] Recepción reconocible.
- [ ] Almacén reconocible.
- [ ] Checkout reconocible.
- [ ] Displays reconocibles y utilizables.
- [ ] Mobiliario alineado y a escala.
- [ ] Productos visibles.
- [ ] Colliders coherentes.
- [ ] NavMesh o navegación actualizada.
- [ ] Puntos de interacción accesibles.
- [ ] Iluminación representativa.
- [ ] Audio representativo mínimo.
- [ ] Sin placeholders visibles no aprobados.
- [ ] Sin referencias funcionales dependientes de nombres frágiles.
- [ ] Capturas y recorrido archivados.

## Anexo C. Fuentes y trazabilidad de consolidación

| Fuente                            | Líneas | Palabras aprox. | SHA-256                                       |
|-----------------------------------|--------|-----------------|-----------------------------------------------|
| Vertical Slice Specification v0.4 | 101    | 355             | efff0d3056515ad3e71a9c751f7ec86ce932d3098fb58 |
| Vertical Slice Specification v0.3 | 109    | 330             | 17213417be3b7b2b95f8e3841d09a6679eee689bb62a3 |
| Vertical Slice Specification v0.2 | 273    | 1403            | 884464e9599f0f1b66fdeff7a3c6422bf9d37b883817f |
| Vertical Slice Specification v0.1 | 258    | 1309            | 69792ba4790022b23d631d437e043d777844f775b94d6 |
| 00_Enfoque_y_Alcance.md           | 4105   | 18925           | 63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f |
| 01_Game_Design_Document.md        | 4400   | 19760           | a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e |

## C.1. Resoluciones principales

- Se adopta el estado funcional de v0.4 como fotografía de referencia.
- Se adopta el Golden Path de v0.4 y se amplía con criterios del GDD consolidado.
- Se conserva la aceptación, el plan de pruebas y la política de severidad de v0.1–v0.2.
- Se conserva la transición de madurez registrada por v0.3.
- Se sustituye la tienda histórica de  $10 \times 10$  celdas por `StoreInitial.unity`, aproximadamente  $10 \times 15$  m.
- Se sustituye el contenido histórico de 12 productos / 6 familias por seis productos iniciales / ocho familias de mobiliario representativas.
- Se distingue la reserva lógica temporal, incluida, del servicio comercial de reservas, diferido.
- Se mantiene la prueba de siete días, los tres slots, ES/EN, los cuatro arquetipos y los ocho clientes como referencias de validación.
- Se separa explícitamente Sprint 16, representativo, de Sprint 17, estabilización y build.
- Se impide interpretar el funcionamiento automatizado como aprobación final sin Golden Path y ejecutable externo.

## Anexo D. Glosario

| Término                     | Definición                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------|
| <b>Acceptance criterion</b> | Condición verificable que debe producir PASS, FAIL u otro resultado controlado.     |
| <b>Build externa</b>        | Ejecutable probado fuera del Unity Editor.                                          |
| <b>Gate</b>                 | Conjunto de condiciones obligatorias para cerrar una fase.                          |
| <b>Golden Path</b>          | Recorrido principal completo y representativo del producto.                         |
| <b>Invariante</b>           | Regla que debe cumplirse antes y después de toda transición válida.                 |
| <b>Mutación atómica</b>     | Cambio que se completa entero o no modifica el estado.                              |
| <b>Placeholder</b>          | Representación provisional no apta por defecto para aprobación visual.              |
| <b>Reserva lógica</b>       | Exclusión temporal de una unidad para impedir dobles ventas.                        |
| <b>Representativo</b>       | Suficiente para evaluar identidad, lectura e integración, aunque no sea arte final. |

| Término               | Definición                                                                                                    |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>S0–S4</b>          | Escala de severidad de defectos definida en este documento.                                                   |
| <b>Soak test</b>      | Prueba prolongada destinada a detectar degradación acumulativa.                                               |
| <b>Vertical slice</b> | Recorrido reducido pero integrado que demuestra experiencia, arquitectura, calidad y capacidad de producción. |

---

**Estado del documento:** contrato vigente para el cierre del vertical slice en la nueva carpeta **Documentacion/**.